Project Summary

Model Name:
Lightweight Hand Gesture Recognition using MediaPipe & Custom MLP (Multi-Layer Perceptron) via TensorFlow Lite.

Objective:
To accurately detect, track, and classify complex, multi-hand Human-Robot Interaction (HRI) scenarios (e.g., Arms waving, Beckoning, Pointing) in real-time, while gracefully handling conflicts between resting hands and active motions.

Data set:
* Static Signs: HaGRID (HAnd Gesture Recognition Image Dataset) - specifically the hagrid-sample-30k-384p subset, consisting of real-world, full-color images to ensure high environmental robustness.
* Dynamic Motions: Custom point-history data collected locally via webcam to match exact target frame rates and environmental conditions.

Techniques used:
* MediaPipe Hand Tracking: Used as a feature extractor to locate 21 hand landmarks per hand, configured for multi-hand tracking (max_num_hands=2).
* Coordinate Normalization: Landmark coordinates are pre-processed and normalized relative to the wrist to ensure the model is invariant to hand size and distance from the camera.
* Custom Neural Networks: Two lightweight MLP networks (one for static shapes, one for motion history) trained using Keras and exported to .tflite for high-speed CPU inference.
* Dynamic Gesture Priority: Algorithmic resolution logic that dictates dynamic movements (like beckoning) automatically override static signs (like resting hands) when two hands are detected.
* Targeted Confidence Thresholding: Applied a strict 85% confidence cutoff exclusively to the 'Pointing' classification to eliminate false positives.
* Decoupled FPS Synchronization: Core camera processing is locked at 15 FPS to ensure accurate temporal motion tracking, while data logging and UI scenario updates are mathematically throttled to exactly 5 FPS using a modulo frame counter.

Extracted features:
* Static Features: A 1D array of 42 normalized float values representing the (x, y) coordinates of the 21 finger joints.
* Dynamic Features: A 1D array of 32 normalized float values representing 16 consecutive frames of movement history for the index finger tip.

Output:
* Visual: Real-time UI overlay rendering the full hand skeleton, bounding boxes, individual hand classifications with confidence percentages, and a resolved "Global HRI Scenario".
* Data: Raw, normalized feature vectors (Keypoint and Point History arrays) printed to the console exactly 5 times per second.

Project Summary

Current progress/results:
The static sign model was successfully trained on 4,000 HaGRID samples, achieving ~93% validation accuracy. The application logic is fully finalized, capable of ingesting both live webcam feeds (app.py) and pre-recorded test videos (test_video.py). It accurately identifies and resolves all 9 specified core HRI scenarios from the project requirements.

Key Challenges Overcome

1. Resting Hand Misclassification:
* Challenge: The Neural Network was forced to classify relaxed, resting hands into one of its trained categories, frequently defaulting to "Pointing". This caused false positive scenarios.
* Solution: Implemented a strict 85% confidence threshold exclusively for the 'Pointing' class. If the model is not highly confident, it falls back to an "Unknown" state.

2. Dynamic vs. Static State Conflicts:
* Challenge: When tracking two hands, a resting hand (falsely identified as a static sign) would mathematically overpower the other hand attempting to perform a dynamic motion (like "Beckoning").
* Solution: Rewrote the global scenario resolution logic so that recognized dynamic actions strictly take precedence over static signs in a two-handed context.

3. Timeline Synchronization (FPS):
* Challenge: The pre-recorded video data required exact 15 FPS processing for the 16-frame motion history math to remain accurate, but the project specifications required data output at exactly 5 FPS. Using basic time delays caused the motion tracking to break.
* Solution: Decoupled the processing loop from the output loop. The camera reads all 15 frames per second, but a modulo frame counter (frame_count % 3 == 0) restricts console printing and UI text updates to exactly 5 times per second.

4. Dataset Format Mismatch:
* Challenge: High-quality internet datasets (like HaGRID) provide raw .jpg images, but the custom MLP model requires normalized 1D coordinate vectors in a .csv format.
* Solution: Engineered a custom data extraction pipeline (extract_dataset.py) that batch-processed thousands of HaGRID images through MediaPipe in the background to generate the exact .csv training files required by the Jupyter Notebooks.